

Paystack Payments for Magento 2

Reference Manual

Version: 3.0.10

Technical reference for developers and system integrators. For installation steps see the Installation Guide; for configuring and operating the extension see the User Guide.

Module Identity

Property	Value
Module name	Pstk_Paystack
PHP namespace	Pstk\Paystack
Composer package	pstk/paystack-magento2-module
Payment method code	pstk_paystack
Method model	Pstk\Paystack\Model\Payment\Paystack

Configuration Paths

All settings are stored under `payment/pstk_paystack/` in Magento configuration.

Path	Admin field	Default
<code>active</code>	Enabled	<code>0</code>
<code>title</code>	Title	Credit/Debit Cards (powered by Paystack)
<code>integration_type</code>	Integration Type	<code>inline</code>
<code>test_mode</code>	Test Mode	<code>1</code>
<code>test_public_key</code>	Test Public Key	—
<code>test_secret_key</code>	Test Secret Key	—
<code>live_public_key</code>	Live Public Key	—
<code>live_secret_key</code>	Live Secret Key	—
<code>debug</code>	Debug	<code>1</code>
<code>sort_order</code>	Sort Order	<code>400</code>
<code>allowspecific</code>	Payment from Applicable Countries	<code>0</code>
<code>specificcountry</code>	Payment from Specific Countries	—

Path	Admin field	Default
<code>min_order_total</code>	Minimum Order Total	<code>100</code>
<code>max_order_total</code>	Maximum Order Total	<code>500000</code>
<code>currency</code>	—	<code>NGN</code>
<code>order_status</code>	—	<code>pending</code>
<code>can_use_checkout</code>	—	<code>1</code>
<code>can_use_internal</code>	—	<code>0</code>

Both secret key fields use Magento's `Magento\Config\Model\Config\Backend\Encrypted` backend model, so values are encrypted at rest and are not readable from the Admin after saving.

`can_use_internal` is `0` deliberately: the method is not offered when creating orders from the Admin.

Storefront Routes

The module registers the front name `paystack` (route id `pstk_paystack`).

Route	Controller	Purpose
<code>/paystack/payment/setup</code>	<code>Controller\Payment\Setup</code>	Initialises a transaction and redirects to Paystack (Redirect flow)
<code>/paystack/payment/callback</code>	<code>Controller\Payment\Callback</code>	Return URL from Paystack; verifies the transaction
<code>/paystack/payment/recreate</code>	<code>Controller\Payment\Recreate</code>	Cancels a failed order, restores the quote, returns to the payment step
<code>/paystack/payment/webhook</code>	<code>Controller\Payment\Webhook</code>	Receives server-to-server events from Paystack

`Controller\Payment\AbstractPaystackStandard` provides shared quote-loading and messaging used by the Redirect-flow controllers.

REST API

Method	Route	Access
GET	<code>/V1/paystack/verify/:reference</code>	anonymous

Serviced by `Pstk\Paystack\Api\PaymentManagementInterface::verifyPayment`. The `reference` segment carries both the Paystack transaction reference and the Magento quote id, joined by an underscore: `{paystackReference}_{quoteId}`.

The endpoint is anonymous because it is called by the checkout JavaScript immediately after the customer completes an inline payment, before any session guarantee exists. Verification is performed server-side against the Paystack API — the client's claim that payment succeeded is never trusted.

Events

Event	Observer	Effect
<code>sales_order_place_before</code>	<code>Observer\ObserverBeforeSalesOrderPlace</code>	Suppresses the initial order confirmation email
<code>paystack_payment_verify_after</code>	<code>Observer\ObserverAfterPaymentVerify</code>	Sets the order to Processing and sends the confirmation email

`paystack_payment_verify_after` is a custom event dispatched by this module. It is the single point at which order status advances and the confirmation email is sent, and it is dispatched by all three verification paths — inline REST verification, Redirect callback, and webhook. Observe it if you need to hook your own logic to confirmed payment.

Webhook

Paystack sends `charge.success` events to `/paystack/payment/webhook` as an HTTP POST.

Each request carries an `X-Paystack-Signature` header containing an HMAC-SHA512 digest of the raw request body, computed with your Paystack secret key. The module validates this signature with a timing-safe comparison before processing, and rejects requests that fail. The transaction is then independently verified against the Paystack API.

Magento's form-key CSRF validation is skipped for this route via `Plugin\CsrfValidatorSkip`, because the request originates from Paystack rather than from a browser session. Signature validation is what authenticates the request.

Content Security Policy

Magento enforces CSP on checkout and payment pages by default from 2.4.7 onward. The module ships `etc/csp_whitelist.xml`, the Magento-standard additive mechanism, which adds the following host sources without replacing your existing policy.

Host	Directive
<code>js.paystack.co</code>	<code>script-src</code> , <code>connect-src</code>
<code>api.paystack.co</code>	<code>connect-src</code>
<code>plugin-tracker.paystackintegrations.com</code>	<code>connect-src</code>

Host	Directive
checkout.paystack.com	frame-src
standard.paystack.co	frame-src

If your store applies a custom CSP outside Magento's mechanism — at a CDN or reverse proxy, for example — these hosts must be allowed there too, or the payment window will be blocked.

Payment Flows

Inline. The order is placed, then checkout JavaScript opens the Paystack window. On success it calls `GET /V1/paystack/verify/{reference}_{quoteId}`. The module verifies with Paystack and dispatches `paystack_payment_verify_after`.

Paystack generates the transaction reference on the client for inline payments, so the quote id is passed as transaction metadata. That gives the webhook a reliable way to locate the order when no Magento-generated reference exists.

Redirect. `/paystack/payment/setup` initialises the transaction and redirects to Paystack. The customer returns to `/paystack/payment/callback`, which verifies the transaction and dispatches `paystack_payment_verify_after`. Failed or abandoned payments route through `/paystack/payment/recreate`.

Webhook. Independent of both, and the reason an order still completes when the customer never returns to the store.

Dependency Injection Scoping

File	Contents
etc/frontend/di.xml	Checkout config provider, <code>PaymentManagerInterface</code> preference, CSRF-skip plugin
etc/webapi_rest/di.xml	<code>PaymentManagerInterface</code> preference for REST calls
etc/adminhtml/di.xml	Intentionally empty
etc/di.xml	Root scope, minimal

`etc/adminhtml/di.xml` is deliberately empty. Registering the frontend bindings in the Admin area caused Adobe Commerce interceptors to fail during Admin order creation. Keep Admin-area DI empty unless you have verified the change against Adobe Commerce, not only Magento Open Source.

Compatibility

Item	Supported
Magento Open Source	2.4.x
Adobe Commerce	2.4.x
PHP	8.2+
Multi-address checkout	Enabled (<code>allow_multiple_address</code>)
Admin order creation	Not supported by design

Support

For issues with this extension, use the [issue tracker](#).

For questions about your Paystack account, send a message from [our website](#).